

Vel Tech Group of Institutions

Name: UMA MAHESHWARI S

Email: vtu28408@veltech.edu.in

Roll no: vtu28408

Phone: null

Branch: VTU

Department: CSE with AIDS

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIDS

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 12_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 24

Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

$\text{sum}(\text{subset elements}) = \text{target}$

Input Format

The first line contains an integer n , representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t , representing the target sum.

Output Format

Print each valid non-empty subset whose elements sum to t , one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

3

Output: 1 2

3

Answer

```
#include <stdio.h>
```

```
int found = 0;
```

```
void findSubsets(int arr[], int n, int index, int target, int currentSum, int subset[], int subsetSize) {
```

```
    // If current sum equals target and subset is not empty
```

```
    if (currentSum == target && subsetSize > 0) {
```

```
        found = 1;
```

```
        for (int i = 0; i < subsetSize; i++) {
```

```
            printf("%d ", subset[i]);
```

```
        }
```

```
        printf("\n");
```

```
        // Do not return; continue to find more subsets
```

```
    }
```

```
    // If reached end
```

```
    if (index == n) {
```

```
        return;
```

```
    }
```

```
    // Include current element
```

```
    subset[subsetSize] = arr[index];
```

```
    findSubsets(arr, n, index + 1, target, currentSum + arr[index], subset, subsetSize + 1);
```

```
    // Exclude current element
```

```
    findSubsets(arr, n, index + 1, target, currentSum, subset, subsetSize);
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int target;
```

```
    scanf("%d", &target);
```

```
int subset[n];

findSubsets(arr, n, 0, target, 0, subset, 0);

if (!found) {
    printf("No subset matches the target sum");
}

return 0;
} // You are using GCC
```

Status : Partially correct

Marks : 4/10

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k .

Input Format

The first line contains two integers n and m , representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v , representing a connection between nodes u and v .

The last line contains an integer k , representing the required clique size.

Output Format

If a clique of size k exists, the output prints: "Clique Found: v1 v2 v3 ..."

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 6

0 1

0 2

1 2

1 3

2 3

3 4

3

Output: Clique Found: 0 1 2

Answer

```
#include <stdio.h>
```

```
int n, m, k;
```

```
int graph[21][21];
```

```
int clique[21];
```

```
int found = 0;
```

```
// Check if vertex v can be added to current clique
```

```
int isSafe(int v, int size) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        if (graph[v][clique[i]] == 0)
```

```
            return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
// Backtracking function
```

```
void findClique(int start, int size) {
```

```
    // If clique size reached
```

```
    if (size == k) {
```

```
        printf("Clique Found: ");
```

```

    for (int i = 0; i < k; i++) {
        printf("%d ", clique[i]);
    }
    found = 1;
    return;
}

```

```

    for (int i = start; i < n; i++) {
        if (isSafe(i, size)) {
            clique[size] = i;
            findClique(i + 1, size + 1);

```

```

        if (found) return; // Stop after first clique
    }
}
}

```

```

int main() {
    scanf("%d %d", &n, &m);

```

```

    // Initialize graph
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            graph[i][j] = 0;

```

```

    int u, v;
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

```

```

    scanf("%d", &k);

```

```

    findClique(0, 0);

```

```

    if (!found) {
        printf("No clique of given size exists");
    }

```

```

    return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

Input Format

The first line of input consists of an integer n , representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

Output Format

If a valid equal partition exists, print:

- Subset 1: x_1 x_2 x_3 ...
- Subset 2: y_1 y_2 y_3 ...

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 5 1 5

Output: Subset 1: 1 5 5

Subset 2: 11

Answer

```
#include <stdio.h>
```

```
int n;  
int arr[20];  
int used[20]; // mark elements in subset1  
int found = 0;
```

```
void findSubset(int index, int currentSum, int target) {  
    if (currentSum == target) {  
        found = 1;  
        return;  
    }
```

```
    if (index == n || found)  
        return;
```

```
    // Include current element  
    used[index] = 1;  
    findSubset(index + 1, currentSum + arr[index], target);
```

```
    if (found) return;
```

```
    // Exclude current element  
    used[index] = 0;  
    findSubset(index + 1, currentSum, target);  
}
```

```
int main() {  
    scanf("%d", &n);
```

```
    int total = 0;  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
        total += arr[i];  
        used[i] = 0;  
    }
```

```
    // If total sum is odd no partition  
    if (total % 2 != 0) {
```



```
    printf("No valid equal partition exists.");
    return 0;
}

int target = total / 2;

findSubset(0, 0, target);

if (!found) {
    printf("No valid equal partition exists.");
    return 0;
}

// Print Subset 1
printf("Subset 1: ");
for (int i = 0; i < n; i++) {
    if (used[i]) {
        printf("%d ", arr[i]);
    }
}

// Print Subset 2
printf(" Subset 2: ");
for (int i = 0; i < n; i++) {
    if (!used[i]) {
        printf("%d ", arr[i]);
    }
}

return 0;
} // You are using GCC
```

Status : Correct

Marks : 10/10